Container Images



DRIM

Open Container Initiative

OCI



The **Open Container Initiative** is an open governance structure for the express purpose of creating open industry standards around container formats and runtimes.

https://opencontainers.org/

OCI Runtime Spec

The Open Container Initiative Runtime Specification aims to specify the configuration, execution environment, and lifecycle of a container.

A container's configuration is specified as the config.json for the supported platforms and details the fields that enable the creation of a container. The execution environment is specified to ensure that applications running inside a container have a consistent environment between runtimes along with common actions defined for the container's lifecycle.

https://github.com/opencontainers/runtime-spec

OCI Image Spec

The Open Container Initiative Image Specification defines an OCI Image, consisting of an i

mage manifest, an *image index* (optional), a set of *filesystem layers*, and a *configuration*.

The goal of this specification is to enable the creation of interoperable tools for building,

transporting, and preparing a container image to run.

https://github.com/opencontainers/image-spec

OCI Distribution Spec

The Open Container Initiative Distribution Specification defines an API protocol to facilitate and standardize the distribution of content.

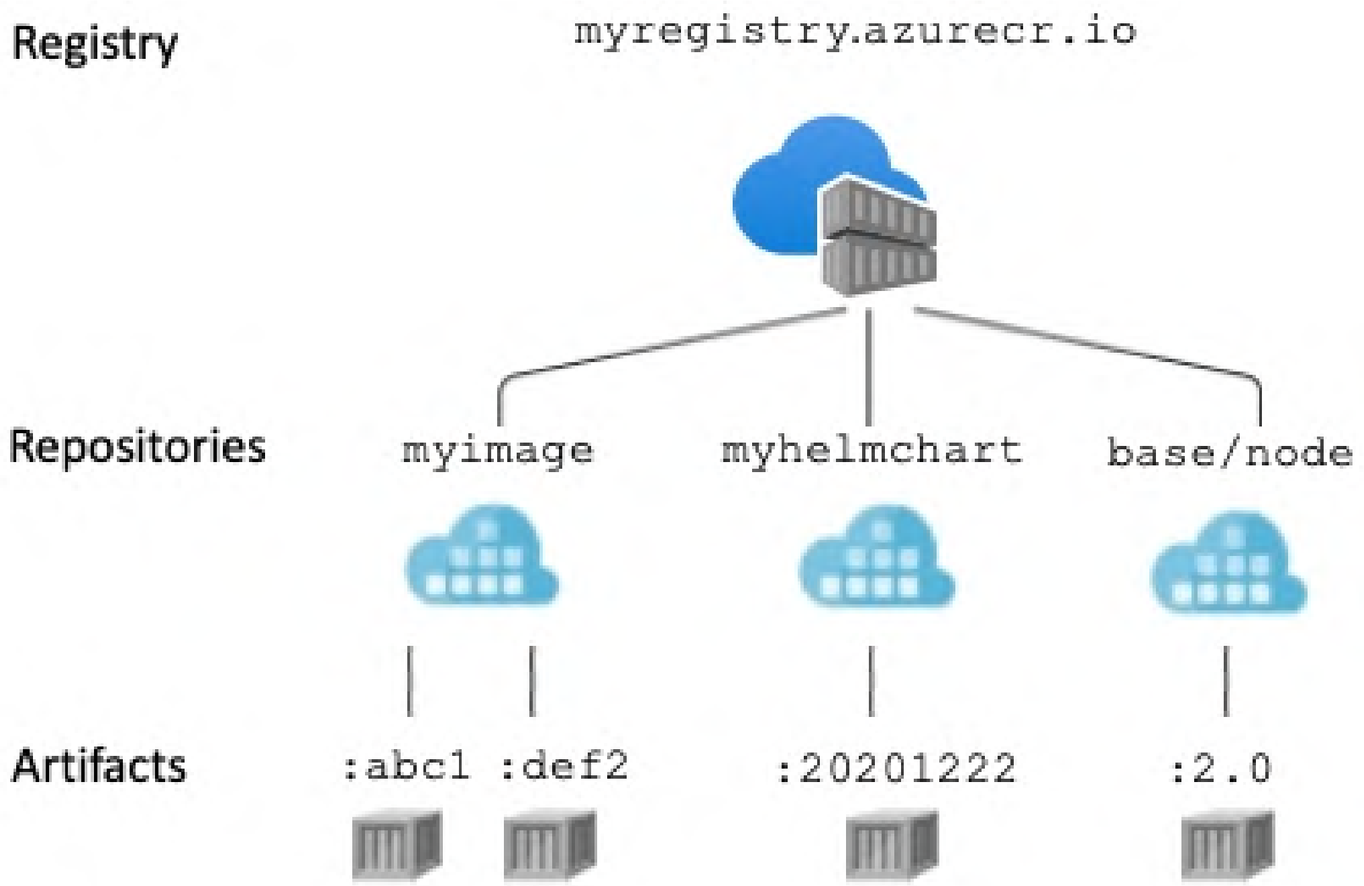
The specification is designed to be agnostic of content types. OCI Image types are currently the most prominent, which are defined in the Open Container Initiative Image Format Specification (a.k.a. "OCI Image Spec").

https://github.com/opencontainers/distribution-spec

DRIM

OCI Distribution

Registries & Repositories



Registries & Repositttories

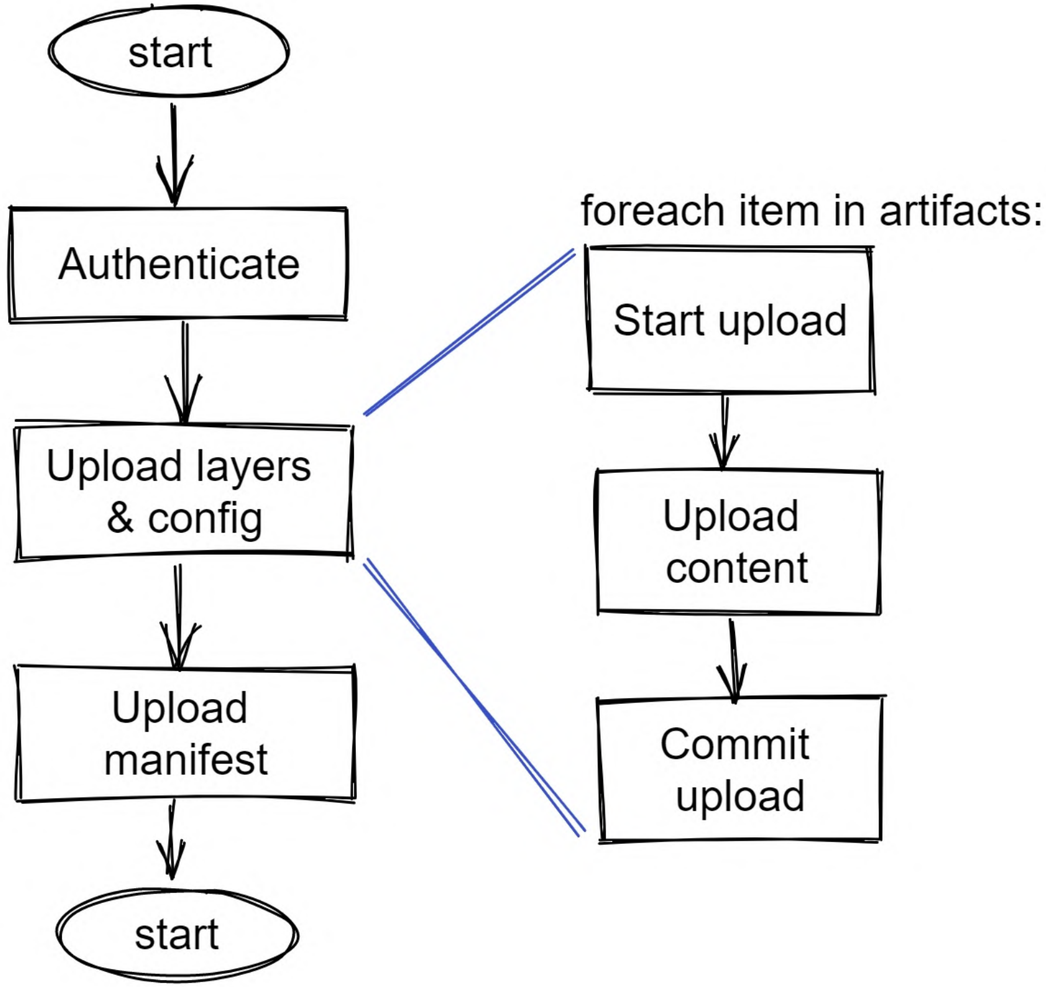
Registry == GitHub.com

Repository == CryptoBank Git Repo

Tags == Git Tags

DRIM

Registry Image Upload



Skopeo



skopeo

https://github.com/containers/skopeo

Skopeo

Skopeo works with API V2 container image registries such as docker.io and quay.io registries, private registries, local directories and local OCI-layout directories. Skopeo can perform operations which consist of:

- Copying an image from and to various storage mechanisms. For example you can copy images from one registry to another, without requiring privilege.
- Inspecting a remote image showing its properties including its layers, without requiring you to pull the image to the host.
- Deleting an image from an image repository.
- Syncing an external image repository to an internal registry for air-gapped deployments.
- When required by the repository, skopeo can pass the appropriate credentials and certificates for authentication.

DRIM

OCI Images

Content-based addressing

Content-based addressing is a method of identifying and retrieving data or information based on its content rather than its location or address.

In content-based addressing, each piece of data is assigned a unique identifier, often called a content-based address or content hash. This identifier is typically generated using a hashing function, such as SHA-256, which computes a fixed-size hash value based on the content of the data. The content hash is deterministic, meaning that the same input data will always produce the same hash value.

Content-based addressing

- It enables data deduplication, as identical data will have the same content hash and can be stored only once, regardless of the number of times it is referenced. This can significantly reduce storage requirements in systems that store large amounts of redundant data
- It provides inherent data integrity. Since the content hash is calculated based on the data itself, any changes to the data will result in a different content hash

Layers

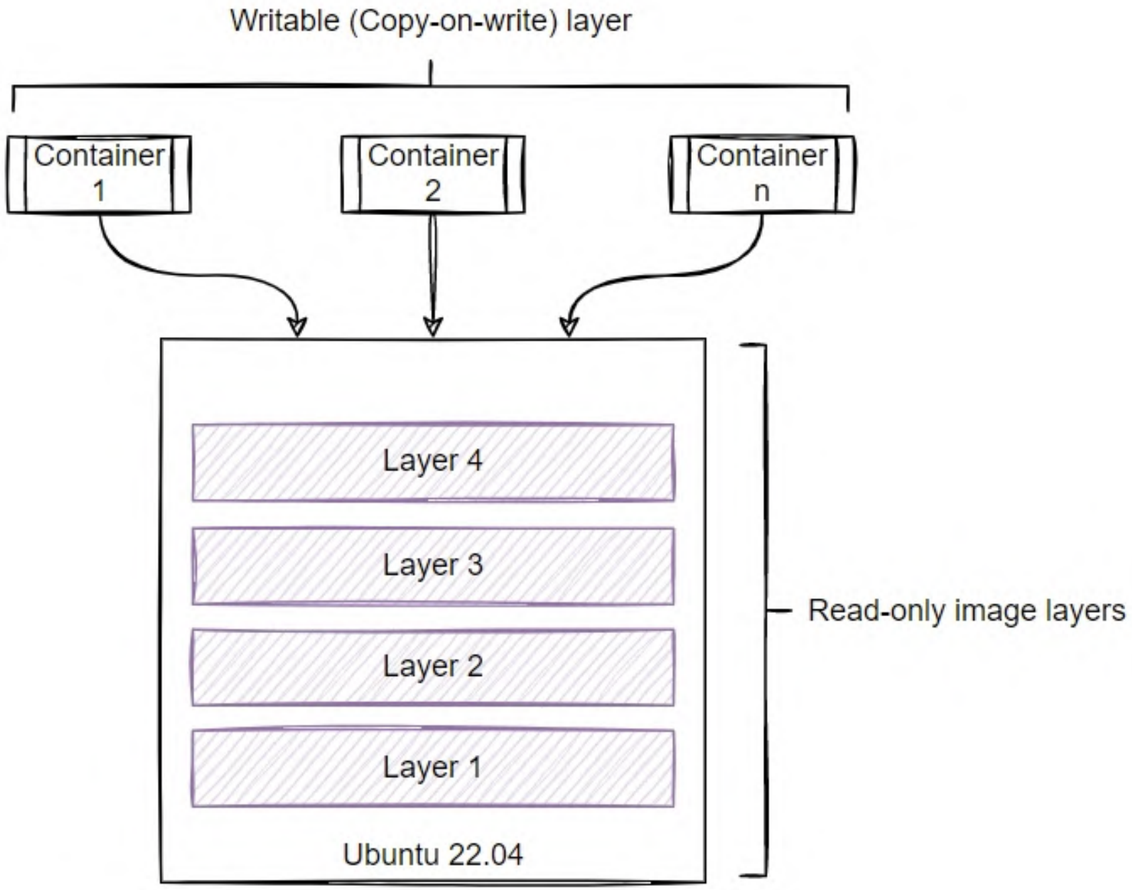

Writable (Copy-on-write) layer
Container 1
Container 2
Container n
Layer 4
Layer 3
Layer 2
Layer 1
Ubuntu 22.04
Read-only image layers

Image Structure

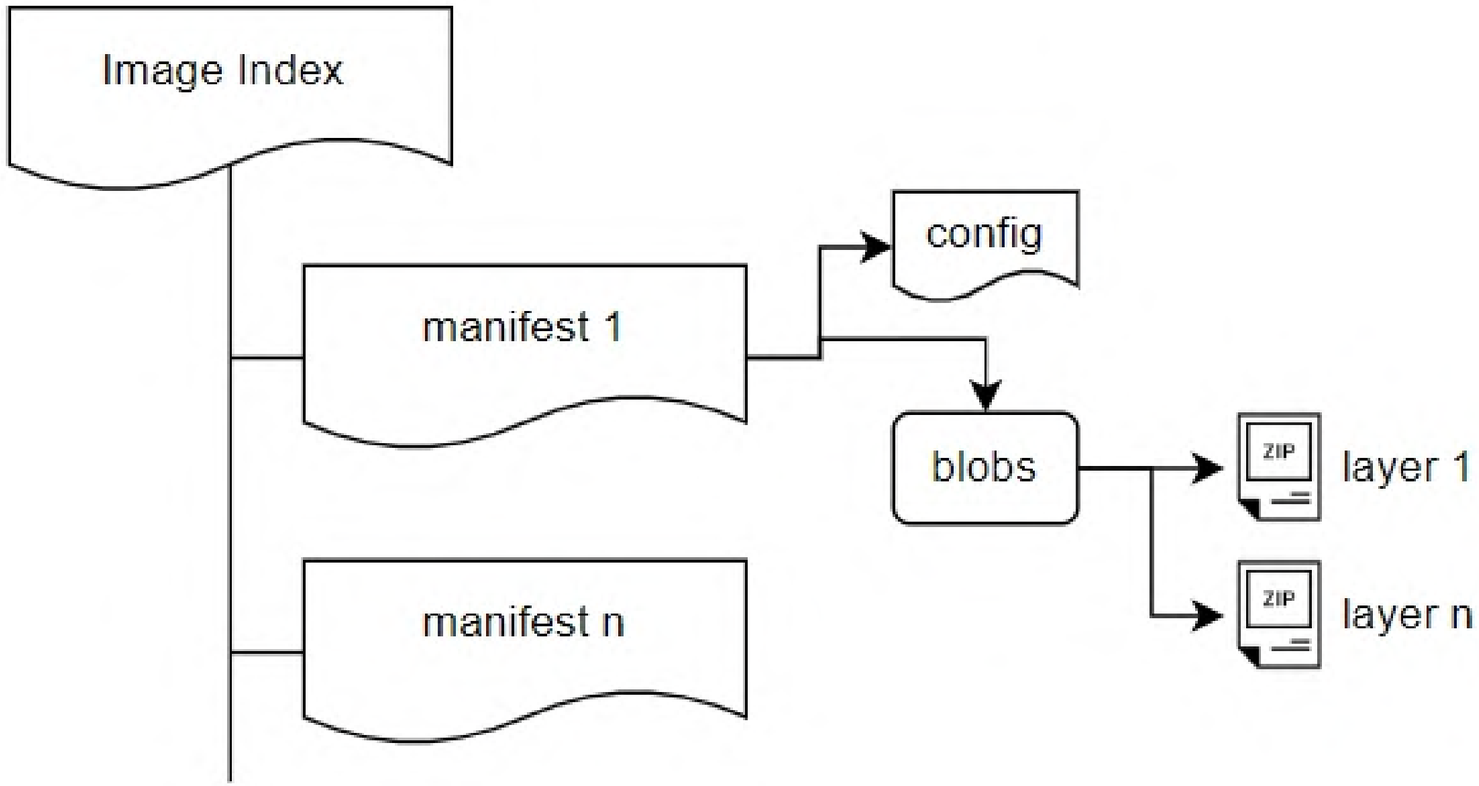


Image Manifest


```
{
    "schemaVersion": 2,
    "mediaType": "application/vnd.oci.image.manifest.v1+json",
    "config": {
      "mediaType": "application/vnd.oci.image.config.v1+json",
      "size": 424,
      "digest": "sha256:8b8ed422c5c3b9ca955067d7d80cbe594d038c4a659a5297db58f09f2decdf5a"
    },
    "layers": [
      {
        "mediaType": "application/vnd.oci.image.layer.v1.tar+gzip",
        "size": 6781394,
        "digest": "sha256:41483ff5b8c5857ae076886a17b82c0861e346c7d2eeadfb42b057f52754cbfb"
      }
    ]
}
```

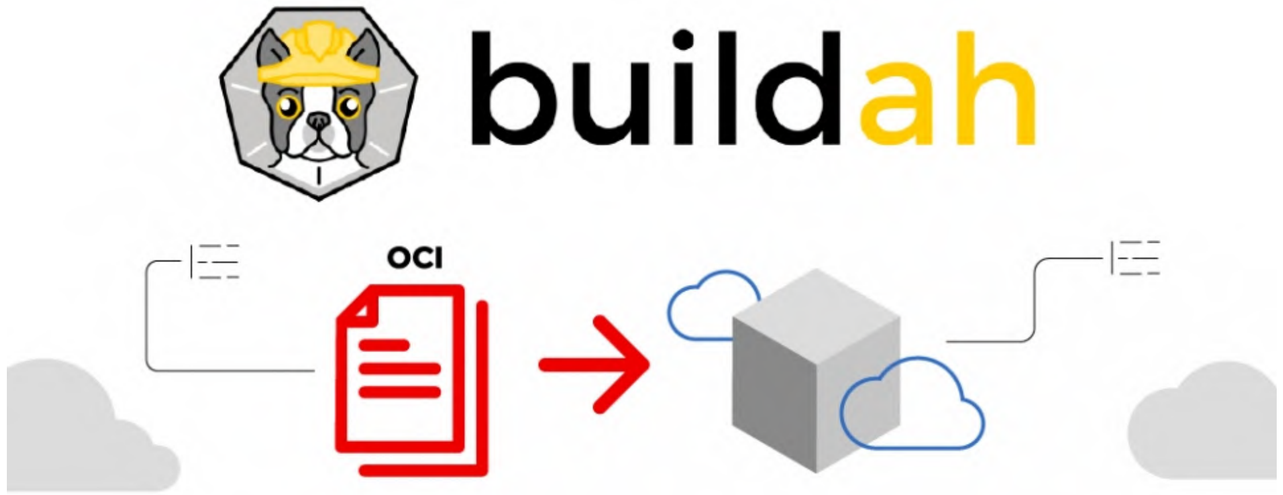
Image Config


```
{
    "architecture": "amd64",
    "os": "linux",
    "config": {
      "Env": [
        "PATH=/app",
        "DOTNET_RUNNING_IN_CONTAINER=true",
        "DOTNET_SYSTEM_GLOBALIZATION_INVARIANT=true"
      ],
      "Entrypoint": [
        "/app/WebApi"
      ]
    },
    "rootfs": {
      "type": "layers",
      "diff_ids": [
        "sha256:39eef564ac2cb0ea0b3fa59b4689951ade1f8e5e208b60b287d31852c077caa8"
      ]
    }
  }
```

containers-storage

```
[/var/lib/containers/storage]$ tree -L 2
.
├── libpod
│   ├── bolt_state.db
│   └── defaultCNINetExists
├── mounts
├── overlay
│   ├── 0cc1f01656262cc1319655e8570146e4aa190c3fb8c7e81c353760c44a96c13b
│   ├── 39c56eae017d464b2d98c351e88eac97ca0bf27cff9b818250e89b39f76bdfc5
│   ├── 586016d81601b62b3d5603da0a5ee0af5d0bab221f4a8ad1f588961fd1d09998
│   ├── 5d847fa1d8bd022f78b57bef0f564c036e9b2899004c1ae6a2a133ffe6c145c9
│   ├── 745673deef16e9347b5a44f9628a1cfd50437b63102d38dd945caf4670d4ae50
│   ├── 7c8dc631fe6cb0c240fd4802cb708b6bcad9ff3f7a76526c5f238838d997f951
│   ├── 7dff8b7d126af5b3a460c0c75800e0aa7a8764f4d3c71e84ae71a29141c65b83
│   ├── d1d58abea916dc9f86fc70e5289a087ce72f17393a82d8e8843750e21aca2633
│   ├── f56b7d23ce34a25718c541313b084b56daa41c9c26a586e480e69bc79bdadbff
│   └── l
├── overlay-containers
│   ├── containers.json
│   └── containers.lock
├── overlay-images
│   ├── 5a8a1bacb12f29e18f2f0b3dcc0a17ed961187e9d360d515c344b3a3f7715c01
│   ├── 87113440139420 7f8866fe15a04f430e533cf30903b2a09587534cdd394cdbfa
│   ├── 94ec3cf46896b8432edad8071a6d2b56ac5c34f6549abb4eb7e6e1544afc1066
│   ├── images.json
│   └── images.lock
├── overlay-layers
│   ├── 0cc1f01656262cc1319655e8570146e4aa190c3fb8c7e81c353760c44a96c13b.tar-split.gz
│   ├── 39c56eae017d464b2d98c351e88eac97ca0bf27cff9b818250e89b39f76bdfc5.tar-split.gz
│   ├── 586016d81601b62b3d5603da0a5ee0af5d0bab221f4a8ad1f588961fd1d09998.tar-split.gz
│   ├── 5d847fa1d8bd022f78b57bef0f564c036e9b2899004c1ae6a2a133ffe6c145c9.tar-split.gz
│   ├── 745673deef16e9347b5a44f9628a1cfd50437b63102d38dd945caf4670d4ae50.tar-split.gz
│   ├── 7c8dc631fe6cb0c240fd4802cb708b6bcad9ff3f7a76526c5f238838d997f951.tar-split.gz
│   ├── 7dff8b7d126af5b3a460c0c75800e0aa7a8764f4d3c71e84ae71a29141c65b83.tar-split.gz
│   ├── d1d58abea916dc9f86fc70e5289a087ce72f17393a82d8e8843750e21aca2633.tar-split.gz
│   ├── f56b7d23ce34a25718c541313b084b56daa41c9c26a586e480e69bc79bdadbff.tar-split.gz
│   ├── layers.json
│   └── layers.lock
├── storage.lock
├── tmp
└── userns.lock
```

Buildah



https://github.com/containers/buildah

Buildah

The Buildah package provides a command line tool that can be used to

- create a working container, either from scratch or using an image as a starting point
- create an image, either from a working container or via the instructions in a Dockerfile
- images can be built in either the OCI image format or the traditional upstream docker image format
- mount a working container's root filesystem for manipulation
- unmount a working container's root filesystem
- use the updated contents of a container's root filesystem as a filesystem layer to create a new image
- delete a working container or an image
- rename a local container

DRIM

Thank you! Questions?

